

RAG or Long Context: How to Get Data Into a Model

Decide between retrieval and stuffing the context window using cost, recall and data size rather than fashion.

For engineers designing the data path into an LLM application · 26 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ Context injection is the shared problem: a frozen model needs current or private data supplied at query time.
- ✓ RAG chunks and embeds documents ahead of time, then retrieves the closest matches per query and injects them.
- ✓ Long context skips the retrieval stack entirely and lets the attention mechanism do the finding.
- ✓ Long context collapses the infrastructure — no chunking strategy, no embedding model, no vector store, no reranker, no sync.
- ✓ RAG's characteristic failure is silent: the answer existed in the data but retrieval never surfaced it, and nothing reports an error.
- ✓ RAG cannot answer questions about what is absent, because it retrieves what exists and never sees the gap between two documents.
- ✓ Long context re-pays the processing cost on every query; RAG pays it once at index time, and prompt caching only helps for static prefixes.
- ✓ Attention dilutes as context grows, so more context is not monotonically better — and enterprise data will never fit regardless.

What it covers

- 01 The problem both approaches solve** 0:00
Models are frozen at their training cutoff and have never seen your private data. Everything else follows from that.
- 02 RAG: the engineering approach** 0:43
Chunk the documents, embed them, store the vectors, search semantically at query time, inject the winners.
- 03 Long context: the model-native approach** 2:10
Skip the database and the embedding model. Put the documents in the prompt and let attention do the heavy lifting.
- 04 What changed: three orders of magnitude** 2:50
Early context windows held about 4K tokens. Today's frontier models handle a million and more, which is what makes the question worth asking.
- 05 The case for long context: no stack, and no retrieval lottery** 4:20
Two arguments. You delete most of the architecture, and you delete the failure mode that never announces itself.
- 06 The whole-book problem: reasoning about what is absent** 5:43
RAG retrieves what exists. Some questions are about what is missing, and no snippet contains a gap.
- 07 The case for RAG: you pay to re-read, every single time** 7:53

Long context creates a compute inefficiency that scales with traffic. RAG pays the processing cost once, at indexing time.

08 **The case for RAG: attention dilutes as context grows** 8:36

There is an intuitive assumption that data in the window will be used. Research suggests otherwise.

09 **The case for RAG: the infinite dataset** 9:19

A million-token window sounds enormous until you compare it to an enterprise data lake measured in terabytes.

10 **Choosing** 10:05

Bounded data plus global reasoning favours long context. Unbounded enterprise knowledge still requires a vector database.

Glossary

Context injection — Getting the right external data into a model's prompt at query time, so it can answer about things outside its training data.

Chunking — Splitting documents into smaller passages before embedding. Strategies include fixed size, sliding window and recursive splitting on structure.

Vector database — Storage built for fast nearest-neighbour search over embeddings, plus the indexing and sync machinery around it.

Silent failure — When retrieval misses a document that contained the answer. No error is raised — the model simply answers from the wrong material, confidently.

Long context — Placing whole documents directly in the prompt and letting the attention mechanism locate what matters, with no retrieval step.

Needle in a haystack — A test that plants a known fact at a controlled depth in a long context and checks whether the model can retrieve it.

recall@k — The fraction of questions for which the correct source passage appears in the top k retrieved results. The ceiling on a RAG system's accuracy.

Context window — The maximum number of tokens a model can process in one call, covering the prompt and the generated output together.

Embedding model — A model that converts text into a vector whose position encodes meaning, so similar passages sit near each other.

Semantic search — Ranking documents by vector similarity to the query rather than by shared keywords, so meaning matches even when wording differs.

Reranker — A second-stage model that re-scores retrieved candidates more accurately than vector similarity, usually over a shortlist.

Prompt caching — Reusing a provider's processed representation of an identical prompt prefix across requests, cutting the cost of the repeated portion.

Attention dilution — The degradation in a model's ability to focus on a specific passage as the surrounding context grows.

Token — The unit a model actually processes — a word fragment. Roughly three-quarters of a word for English prose, and much denser for code or JSON.

Explanations are AI-generated. Check anything you intend to rely on.